TERRAFORM FINAL TASK

1. Create VPC

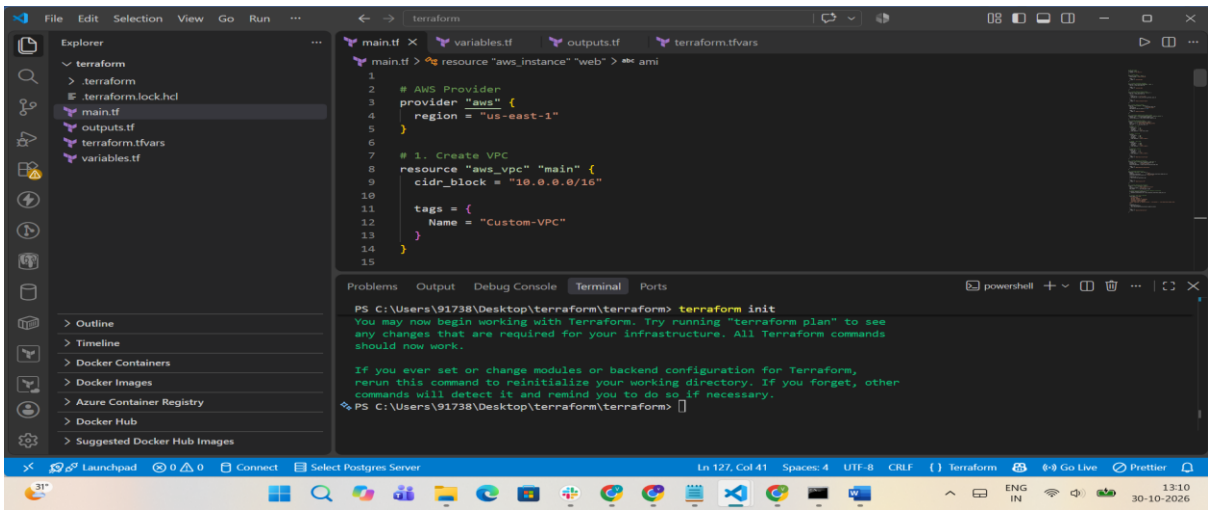
Implementation steps:

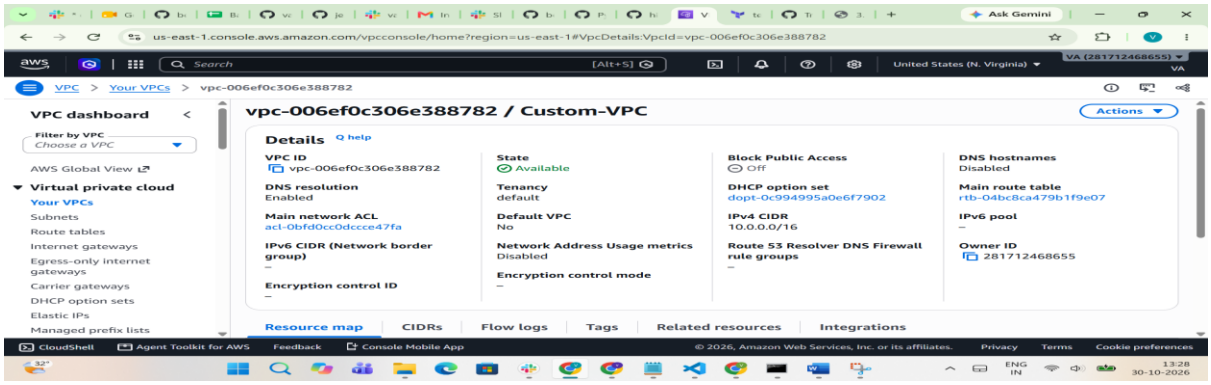
1. Open the Terraform project folder.
2. Create a file named main.tf.
3. Configure the AWS provider with the US.Virginia region (us-east-1).
4. Create a VPC with the CIDR block 10.0.0.0/16.
5. Enable DNS support and DNS hostnames.
6. Run terraform init and terraform apply to create the VPC.

Terraform code:

```
resource "aws_vpc" "main" {
  cidr_block         = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true


  tags = {
    Name = "Custom-VPC"
  }
}
```





2. Create Internet Gateway

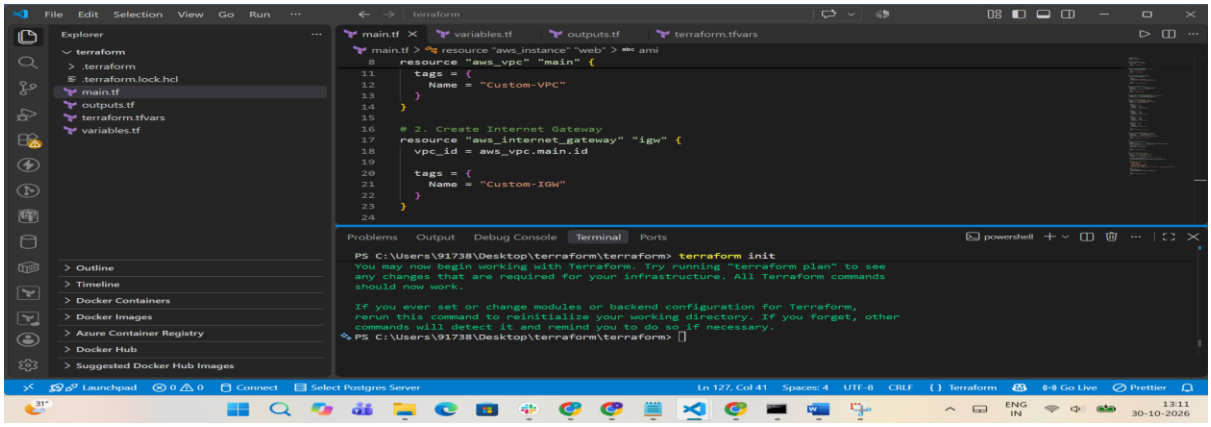
Implementation steps:

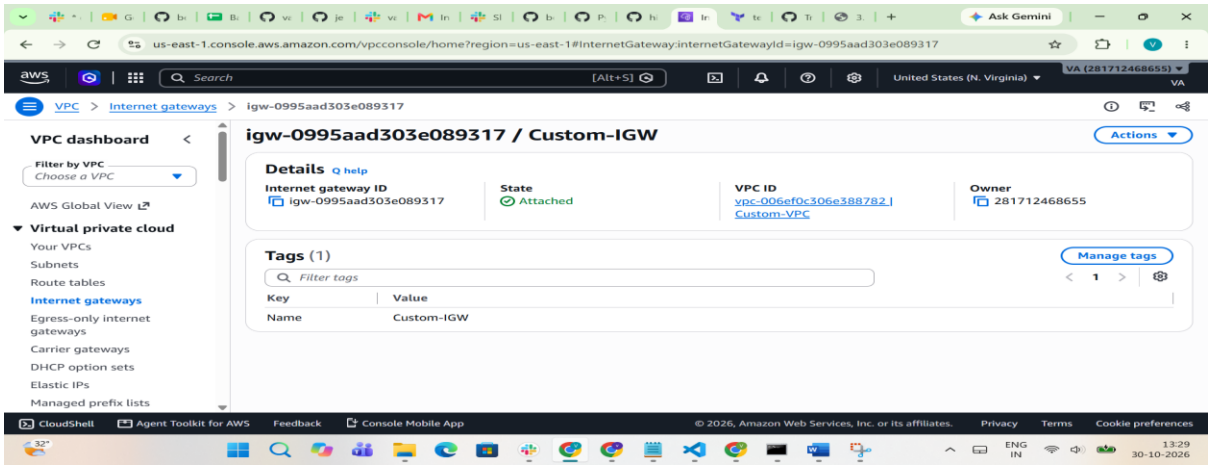
1. Create an Internet Gateway resource in Terraform.
2. Attach the Internet Gateway to the VPC created in Step 1.
3. Add a Name tag to identify the Internet Gateway.
4. Apply the Terraform configuration.
5. Verify that the Internet Gateway is attached to the custom VPC.

Terraform code:

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id


  tags = {
    Name = "Custom-IGW"
  }
}
```





3. Create Custom Route Table

Implementation steps:

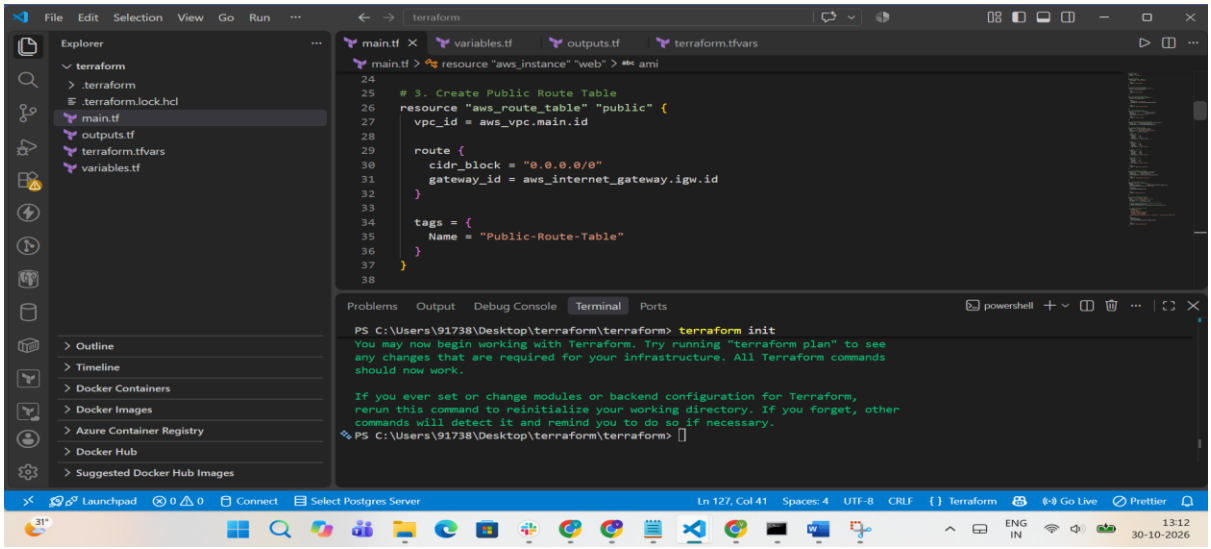
1. Create a custom route table inside the VPC.
2. Add a default route (0.0.0.0/0) pointing to the Internet Gateway.
3. Add a Name tag to the route table.
4. Apply the Terraform configuration.
5. Verify that the route table contains the default route to the Internet Gateway.

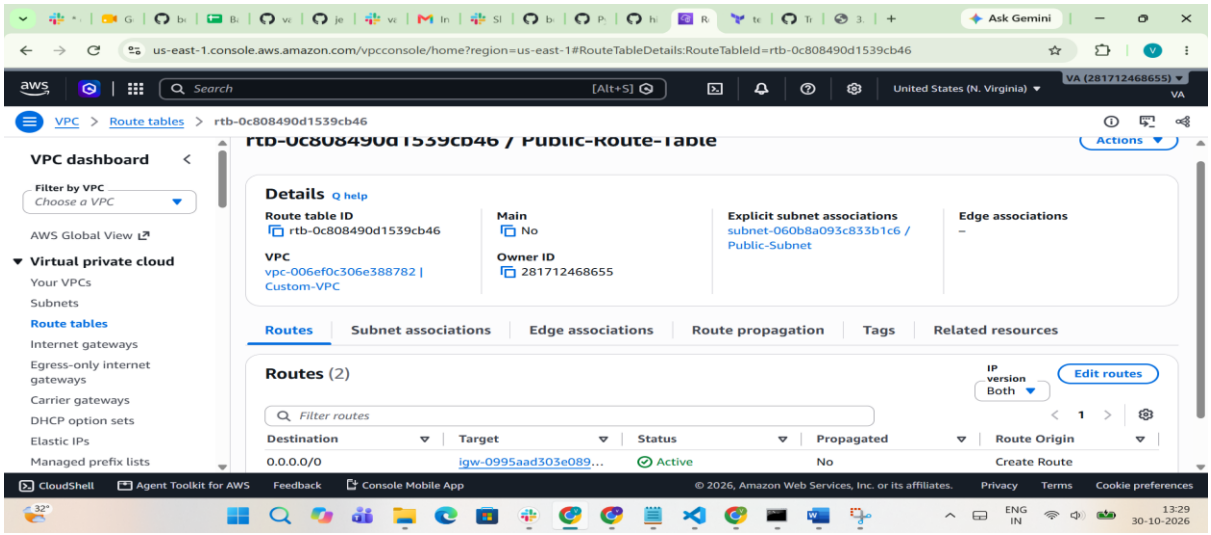
Terraform code:

```
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id


  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }


  tags = {
    Name = "Public-Route-Table"
  }
}
```





4. Create Subnet

Implementation steps:

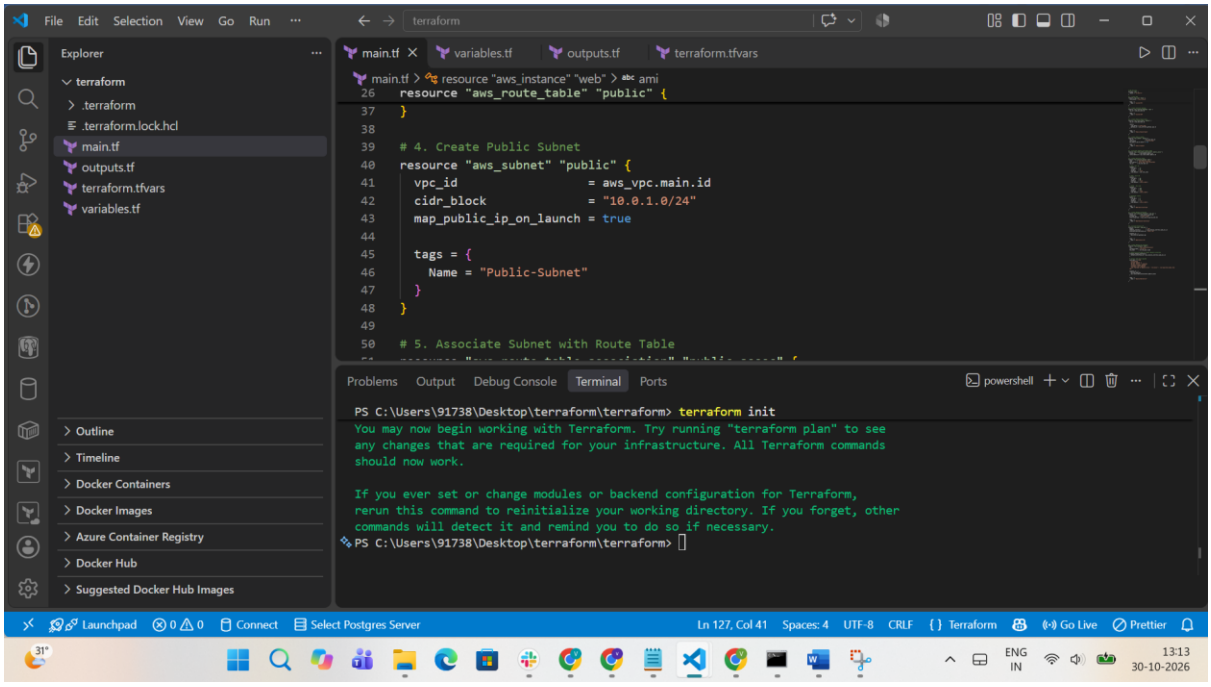
1. Create a subnet inside the custom VPC.
2. Configure the subnet CIDR block as 10.0.1.0/24.
3. Select an Availability Zone, for example, ap-south-1a.
4. Enable auto-assignment of public IPv4 addresses.
5. Apply the Terraform configuration.
6. Verify that the subnet is created inside the custom VPC.

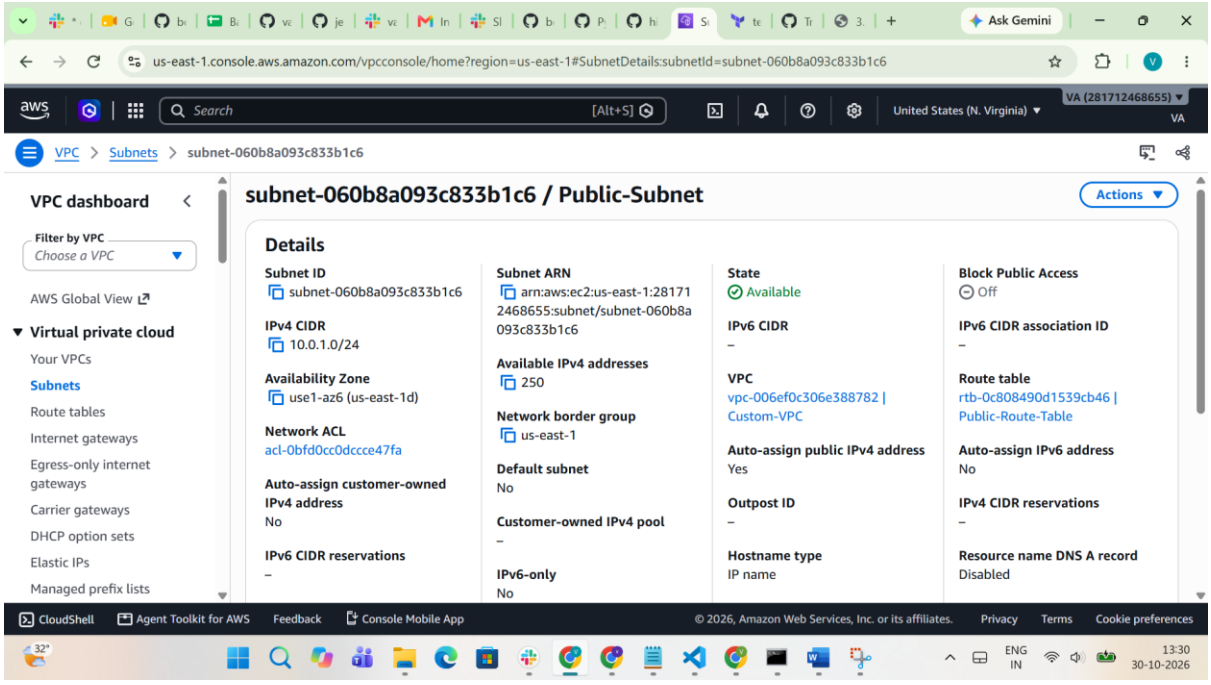
Terraform code:

```
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
```

```
  map_public_ip_on_launch = true


  tags = {
    Name = "Public-Subnet"
  }
}
```





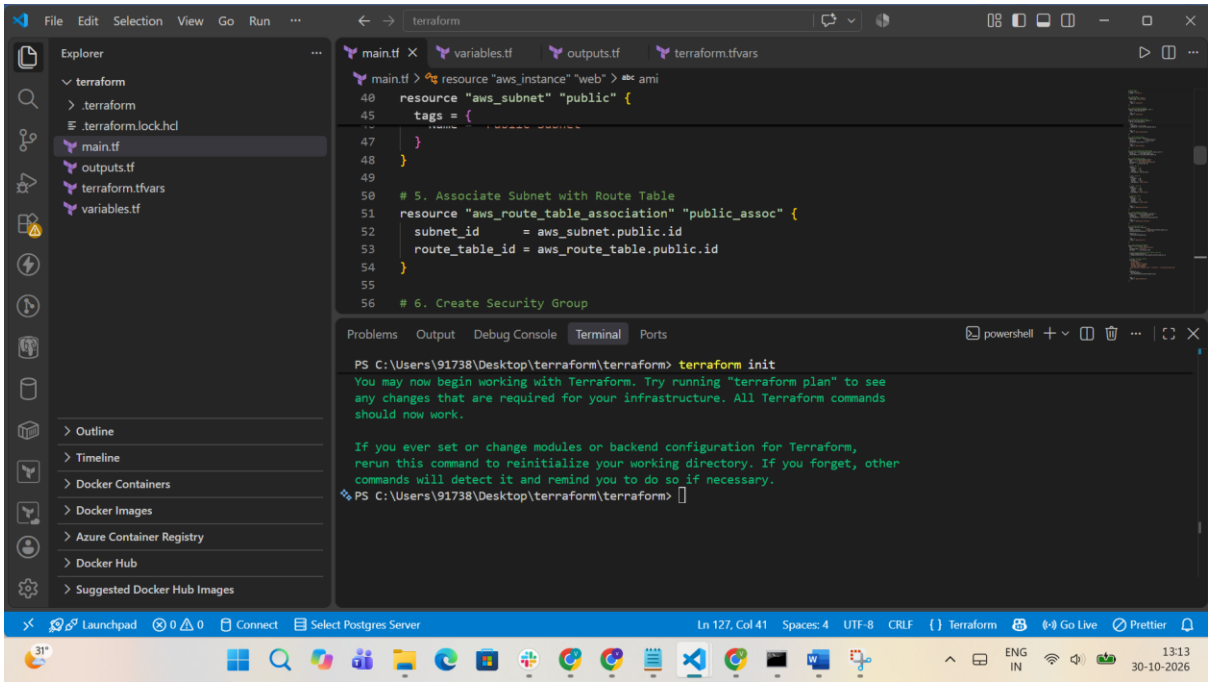
5. Associate Subnet with Route Table

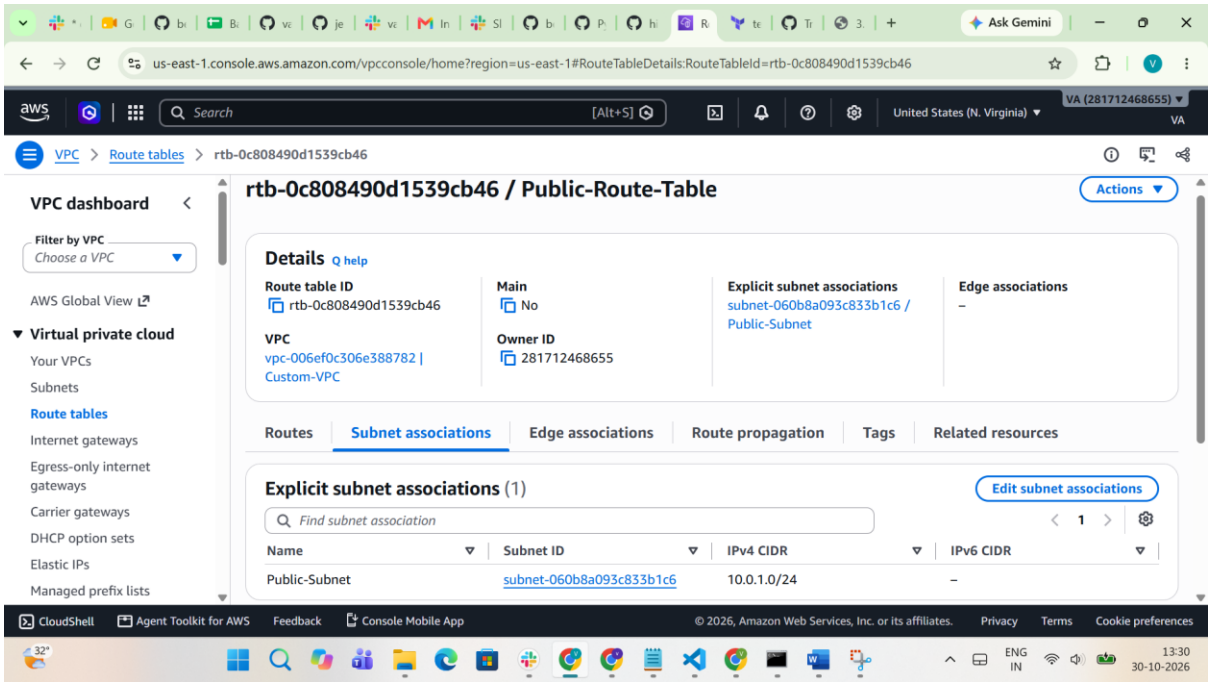
Implementation steps:

1. Create a route table association resource.
2. Select the subnet created in Step 4.
3. Associate it with the custom route table created in Step 3.
4. Apply the Terraform configuration.
5. Verify the subnet association in the Route Tables section.

Terraform code:

```
resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}
```





6. Create Security Group to Allow Ports 22, 80, and 443

Implementation steps:

1. Create a security group inside the custom VPC.
2. Add an inbound rule for SSH (port 22).
3. Add an inbound rule for HTTP (port 80).
4. Add an inbound rule for HTTPS (port 443).
5. Configure outbound traffic to allow all traffic.
6. Apply the Terraform configuration.
7. Verify the inbound and outbound rules in the AWS Security Groups section.

Terraform code:

```
resource "aws_security_group" "web_sg" {
  name        = "Web-Security-Group"
  description = "Allow SSH, HTTP and HTTPS"
  vpc_id      = aws_vpc.main.id


  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
```

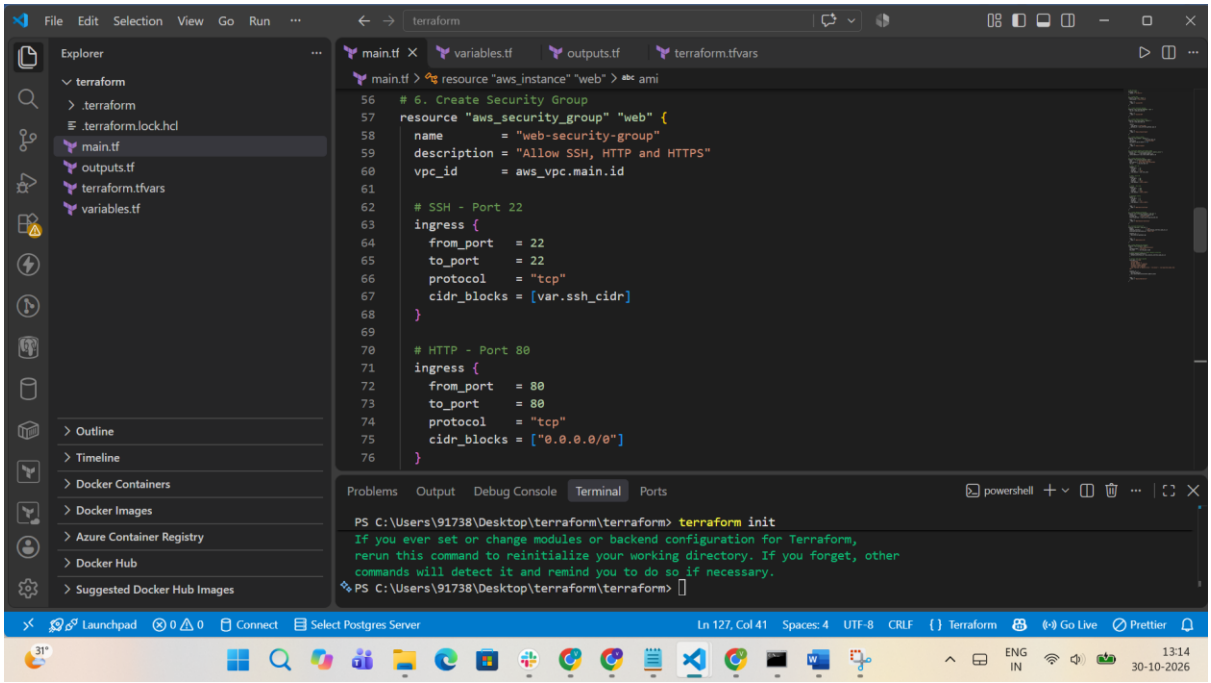
```
  protocol    = "tcp"
  cidr_blocks = [var.ssh_cidr]
 }

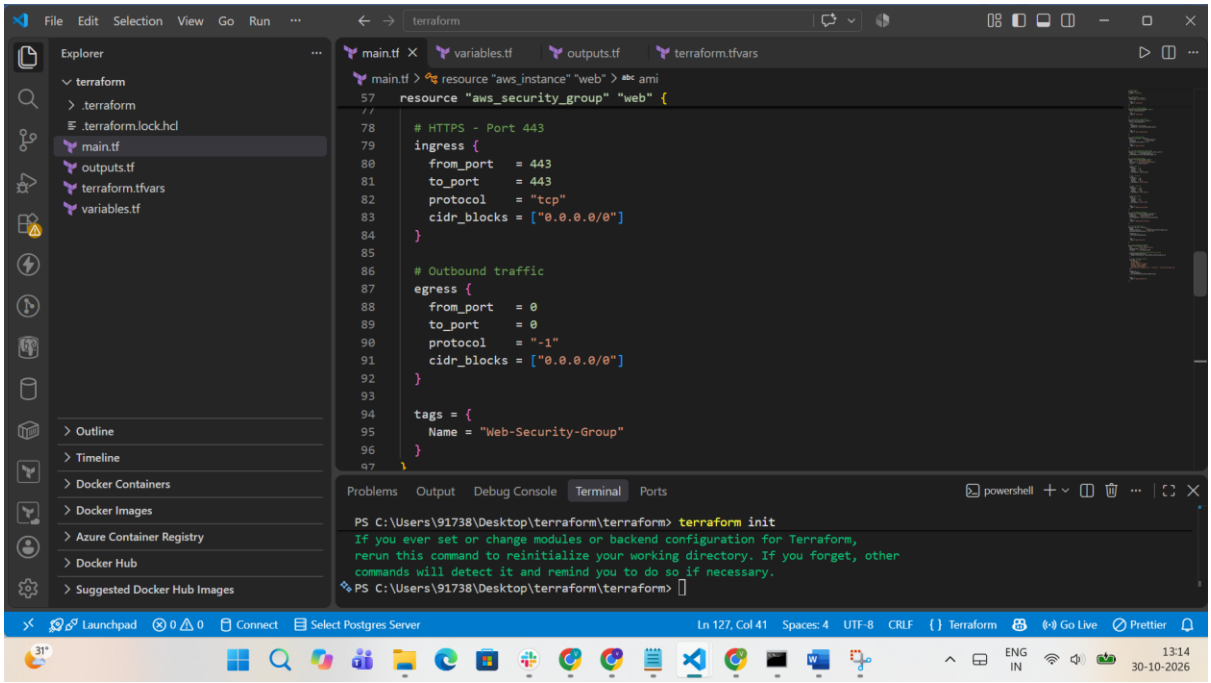
 ingress {
  description = "HTTP"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

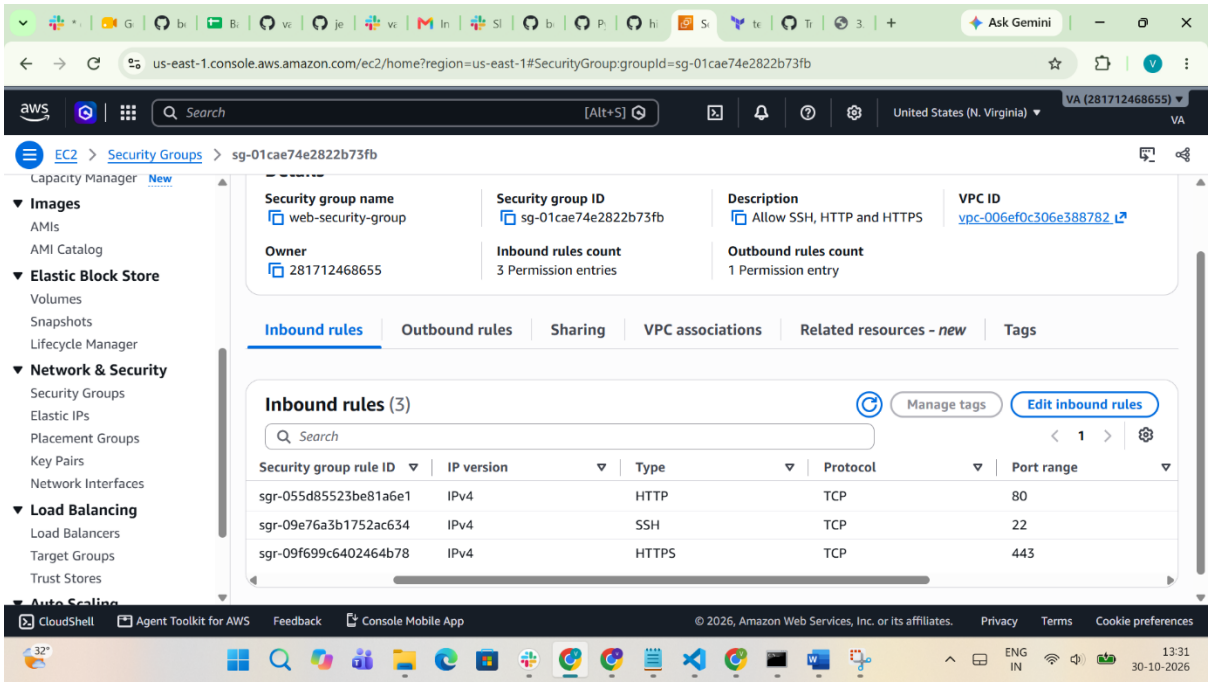
 ingress {
  description = "HTTPS"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
 }

 egress {
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
 }

 tags = {
  Name = "Web-Security-Group"
 }
}
```


File Edit Selection View Go Run
terraform
Explorer
terraform
.terraform
.terraform.lock.hcl
main.tf
outputs.tf
terraform.tfvars
variables.tf
main.tf
variables.tf
outputs.tf
terraform.tfvars
main.tf > resource "aws_instance" "web" > ami
56 # 6. Create Security Group
57 resource "aws_security_group" "web" {
58 name = "web-security-group"
59 description = "Allow SSH, HTTP and HTTPS"
60 vpc_id = aws_vpc.main.id
61
62 # SSH - Port 22
63 ingress {
64 from_port = 22
65 to_port = 22
66 protocol = "tcp"
67 cidr_blocks = [var.ssh_cidr]
68 }
69
70 # HTTP - Port 80
71 ingress {
72 from_port = 80
73 to_port = 80
74 protocol = "tcp"
75 cidr_blocks = ["0.0.0.0/0"]
76 }
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform\terraform> terraform init
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\91738\Desktop\terraform\terraform>
Launchpad 0 0 Connect Select Postgres Server
Ln 127, Col 41 Spaces: 4 UTF-8 CRLF { } Terraform Go Live Prettier
ENG IN
13:14
30-10-2026


File Edit Selection View Go Run
terraform
Explorer
terraform
.terraform
.terraform.lock.hcl
main.tf
outputs.tf
terraform.tfvars
variables.tf
main.tf
variables.tf
outputs.tf
terraform.tfvars
main.tf > resource "aws_instance" "web" > ami
57 resource "aws_security_group" "web" {
78 # HTTPS - Port 443
79 ingress {
80 from_port = 443
81 to_port = 443
82 protocol = "tcp"
83 cidr_blocks = ["0.0.0.0/0"]
84 }
85
86 # Outbound traffic
87 egress {
88 from_port = 0
89 to_port = 0
90 protocol = "-1"
91 cidr_blocks = ["0.0.0.0/0"]
92 }
93
94 tags = {
95 Name = "Web-Security-Group"
96 }
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform\terraform> terraform init
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\91738\Desktop\terraform\terraform>
Launchpad 0 0 Connect Select Postgres Server
Ln 127, Col 41 Spaces: 4 UTF-8 CRLF { } Terraform Go Live Prettier
ENG IN
13:14
30-10-2026



Note: Replace var.ssh_cidr with your public IP address in /32 format through the Terraform variable. Avoid opening SSH to everyone.

7. Create a Network Interface with an IP in the Created Subnet

Implementation steps:

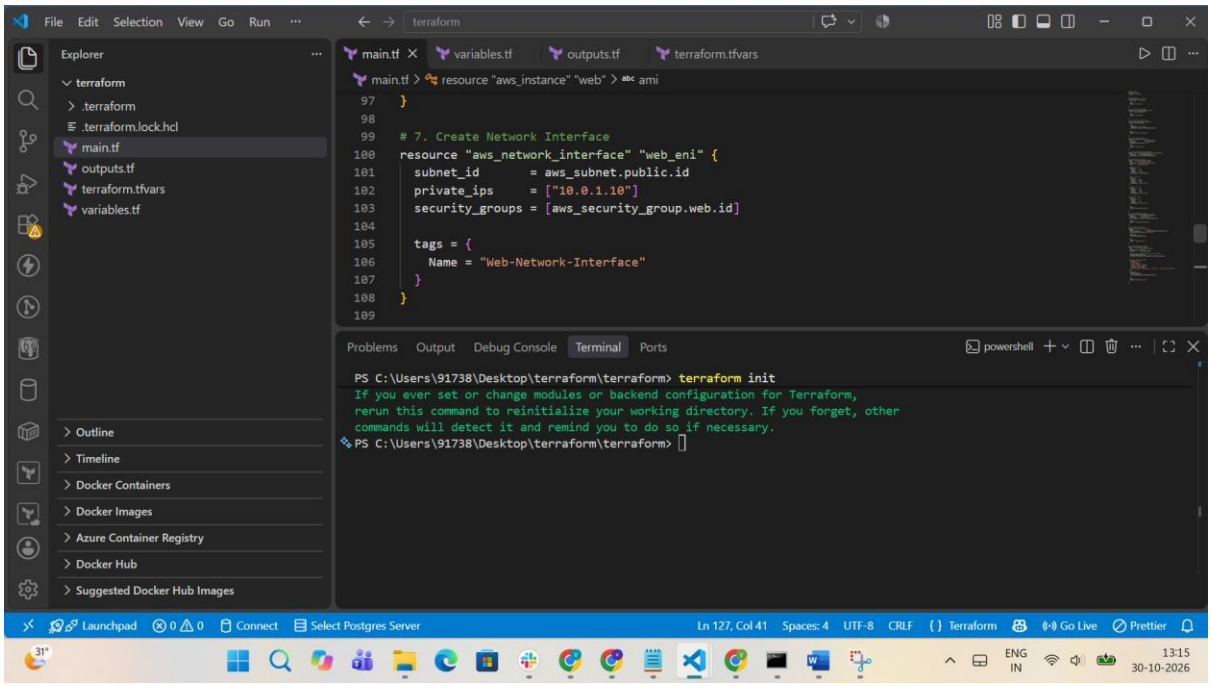
1. Create a network interface using Terraform.
2. Select the subnet created in Step 4.
3. Assign a private IP address, such as 10.0.1.10.
4. Attach the Security Group created in Step 6.
5. Apply the Terraform configuration.
6. Verify the network interface in the EC2 Network Interfaces section.

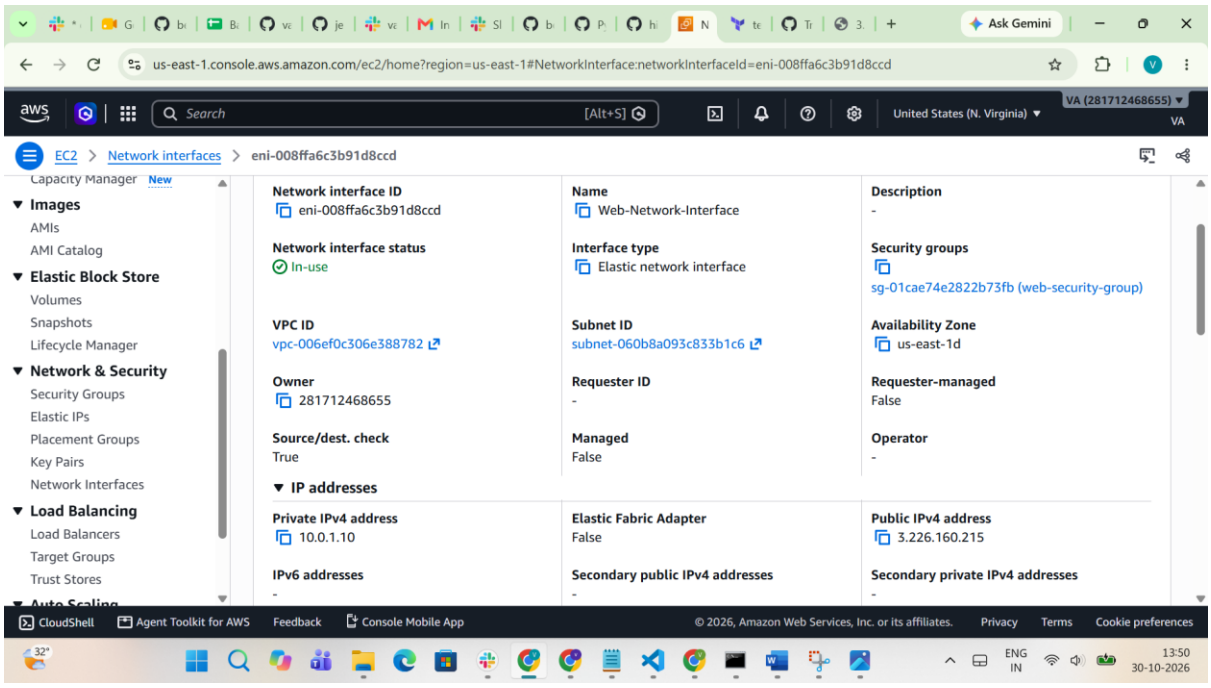
Terraform code:

```
resource "aws_network_interface" "web_eni" {
  subnet_id       = aws_subnet.public_subnet.id
  private_ips     = ["10.0.1.10"]
  security_groups = [aws_security_group.web_sg.id]


  tags = {
    Name = "Web-Network-Interface"
```

```
  }
}
```





8. Assign an Elastic IP to the Network Interface

Implementation steps:

1. Create an Elastic IP resource.
2. Select the VPC domain.

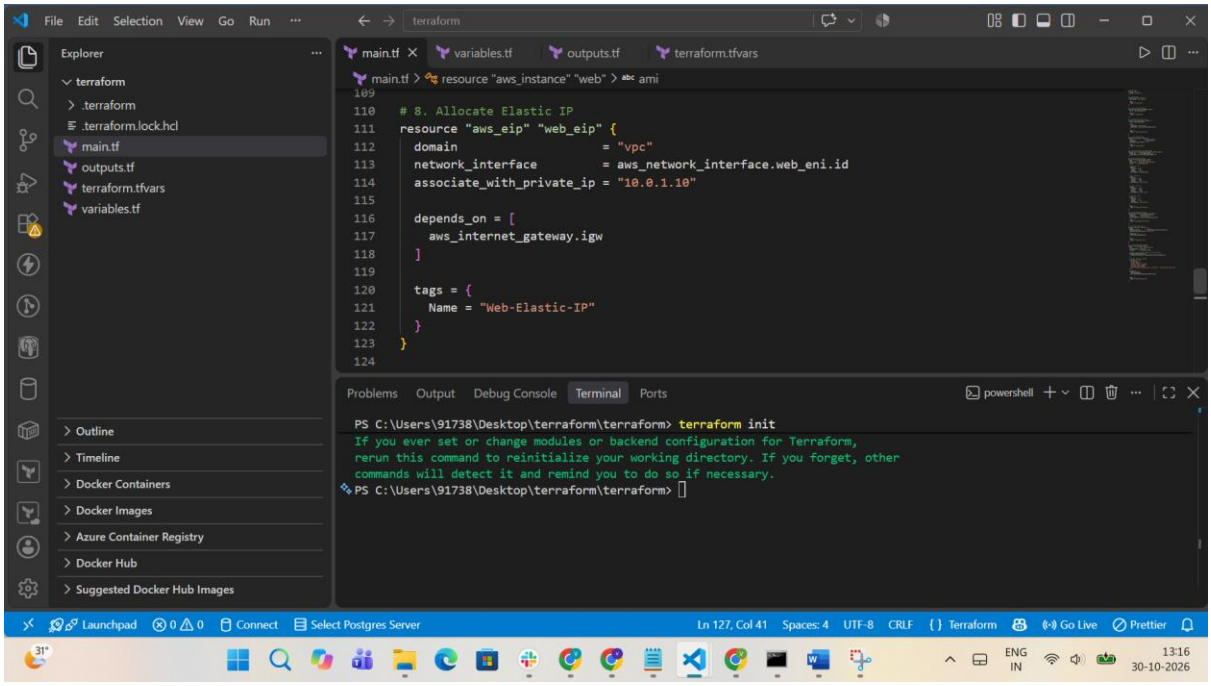
3. Associate the Elastic IP with the network interface created in Step 7.
4. Specify the private IP address 10.0.1.10.
5. Apply the Terraform configuration.
6. Verify that the Elastic IP is associated with the correct network interface.

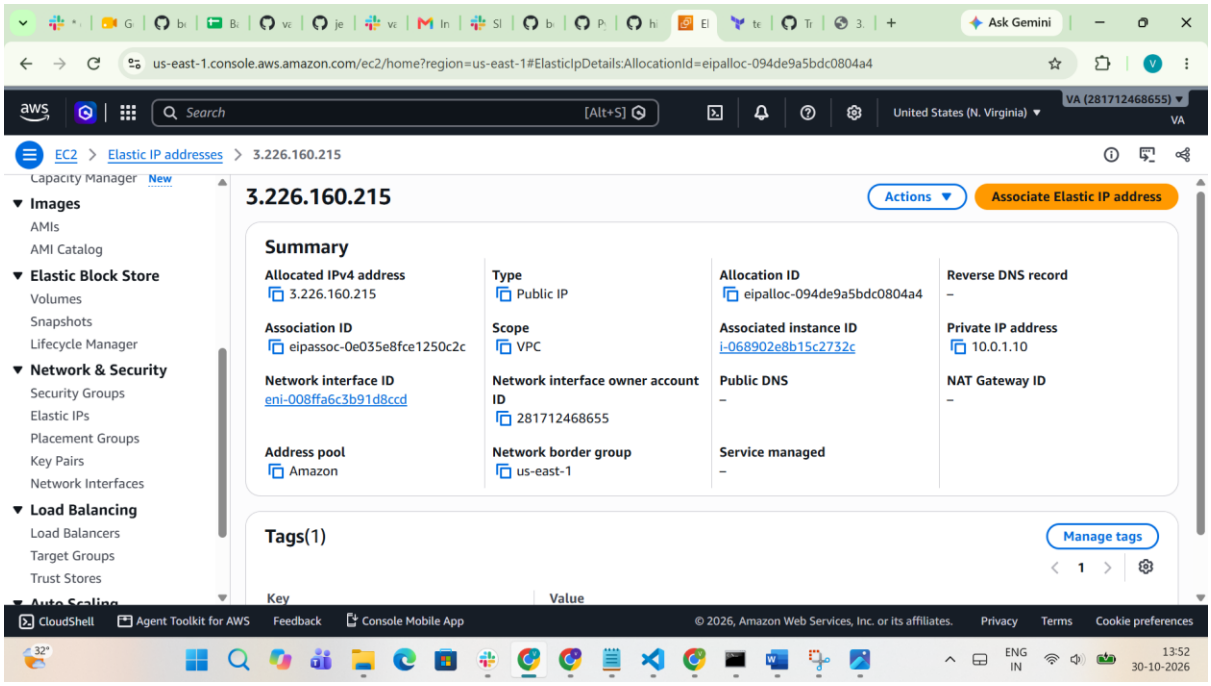
Terraform code:

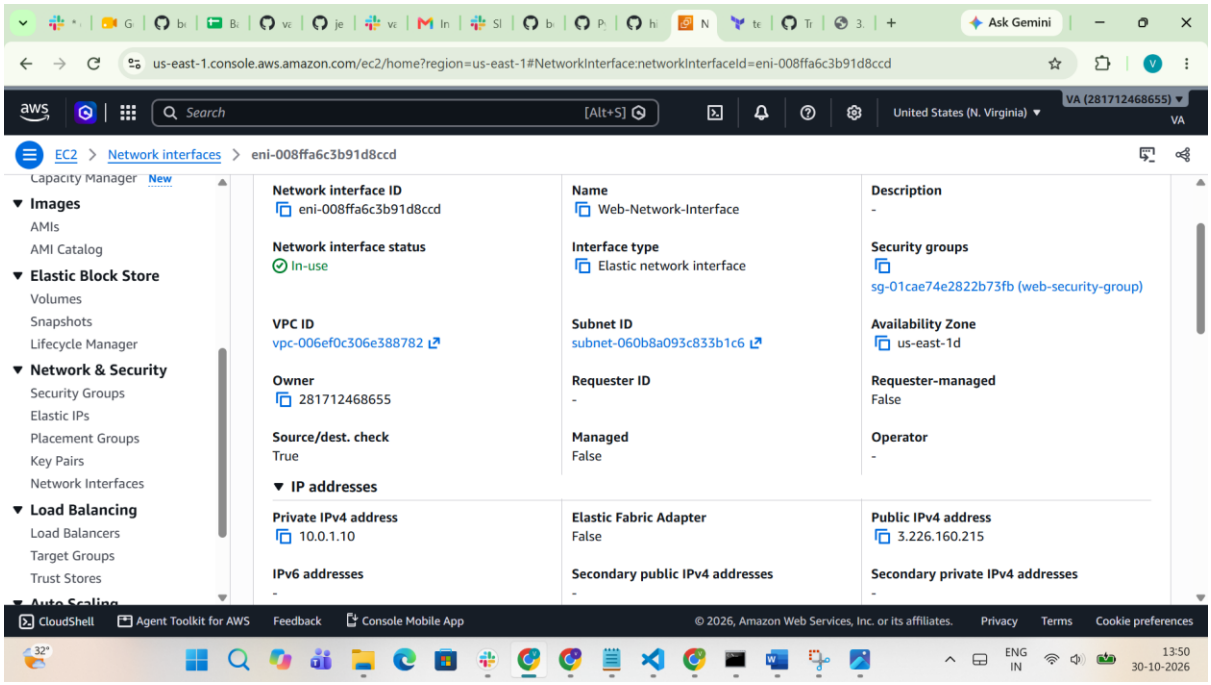
```
resource "aws_eip" "web_eip" {
 domain                   = "vpc"
 network_interface        = aws_network_interface.web_eni.id
 associate_with_private_ip = "10.0.1.10"

 depends_on = [
  aws_internet_gateway.igw
 ]

 tags = {
  Name = "Web-Elastic-IP"
 }
}
```







9. Create Ubuntu Server and Install/Enable Apache2

Implementation steps:

1. Create an EC2 instance using the existing Ubuntu AMI ID from the Mumbai region.
2. Select an instance type, such as t3.micro.
3. Select an existing EC2 key pair.
4. Attach the network interface created in Step 7 as the primary network interface.
5. Configure user data to install Apache2.

6. Enable and start the Apache2 service.
7. Apply the Terraform configuration.
8. Verify that the EC2 instance is running.
9. Copy the Elastic IP address and open it in a browser using http://ELASTIC-IP.
10. Verify that the Apache2 default web page is displayed.

Terraform code:

For the EC2 resource, use the primary network interface configuration supported by your installed AWS provider version. The network_interface block has been marked deprecated in some provider versions, so check your provider schema before using it.

```
resource "aws_instance" "ubuntu_server" {

 ami           = var.ubuntu_ami_id

 instance_type = "t3.micro"

 key_name      = var.key_pair_name


 network_interface {

  network_interface_id = aws_network_interface.web_eni.id

  device_index         = 0

 }


 user_data = <<-EOF

          #!/bin/bash

          apt-get update -y

          apt-get install apache2 -y

          systemctl enable apache2

          systemctl start apache2

          EOF

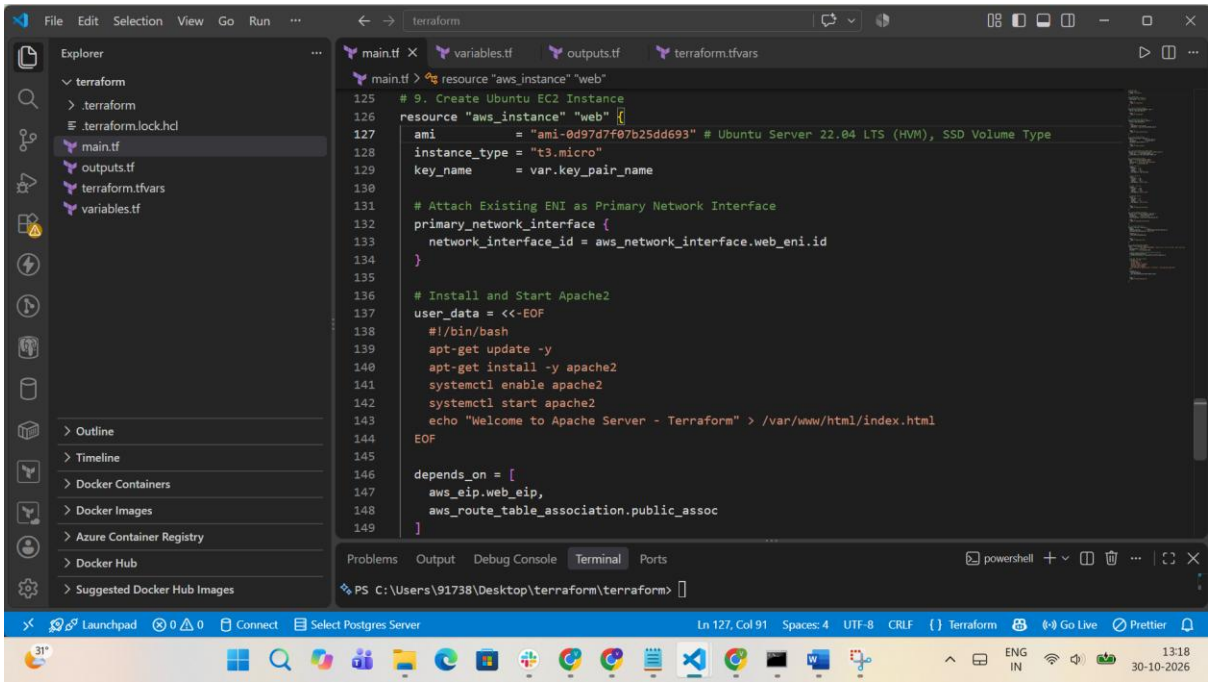

 tags = {

  Name = "Ubuntu-Web-Server"

 }


 depends_on = [
```

```
    aws_eip.web_eip,
    aws_route_table_association.public_assoc
  ]
}
```



Supporting Terraform configuration

Create the following files in your project folder.

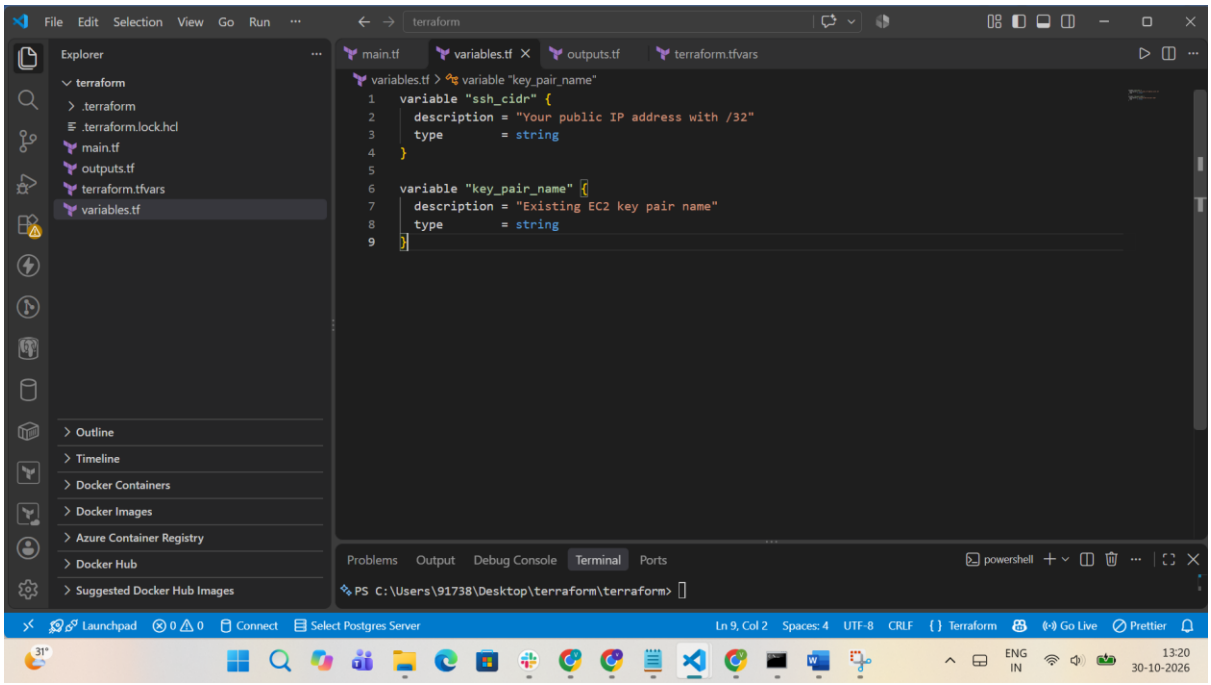
Folder structure:

```
terraform/
|
 ├── main.tf
 ├── variables.tf
 ├── terraform.tfvars
 ├── outputs.tf
```

variables.tf

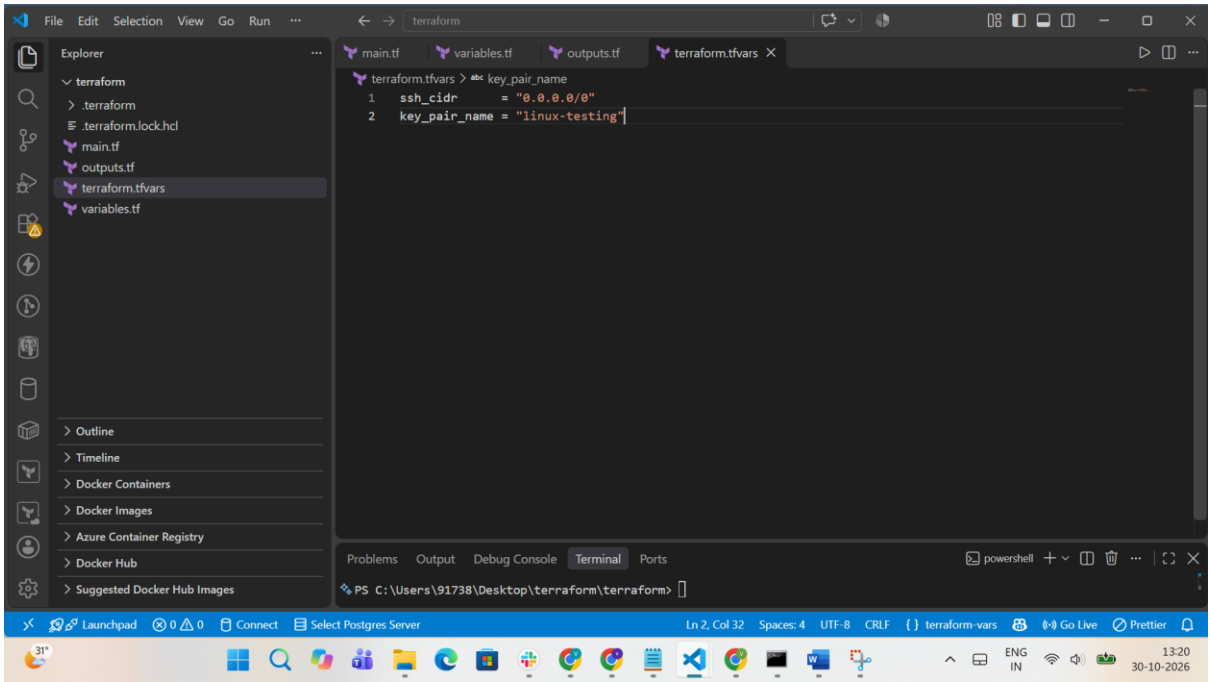
```
variable "ssh_cidr" {
  description = "Your public IP address with /32"
  type        = string
}
```

```
variable "key_pair_name" {
  description = "Existing EC2 key pair name"
  type        = string
}
```



terraform.tfvars

```
ssh_cidr      = "YOUR_PUBLIC_IP/32"
key_pair_name = "your-key-pair-name"
```

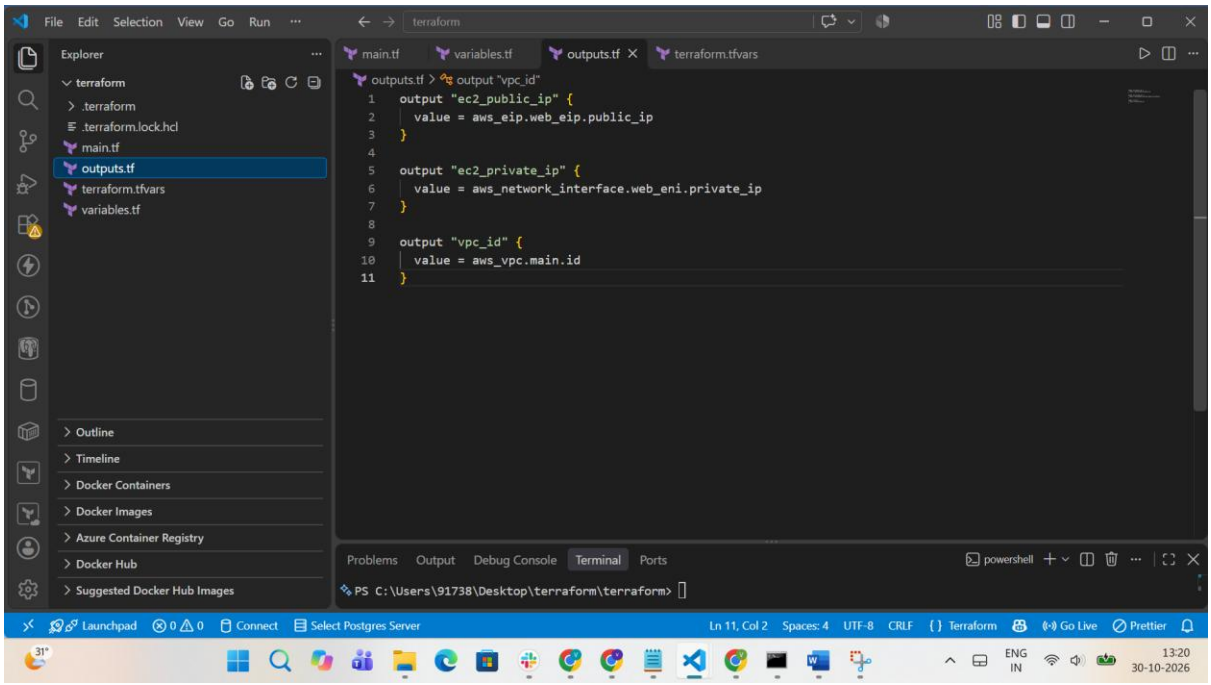


outputs.tf

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "elastic_ip" {
  value = aws_eip.web_eip.public_ip
}

output "private_ip" {
  value = aws_network_interface.web_eni.private_ip
}
```



Terraform execution commands

Run these commands from the project directory:

terraform init

terraform validate

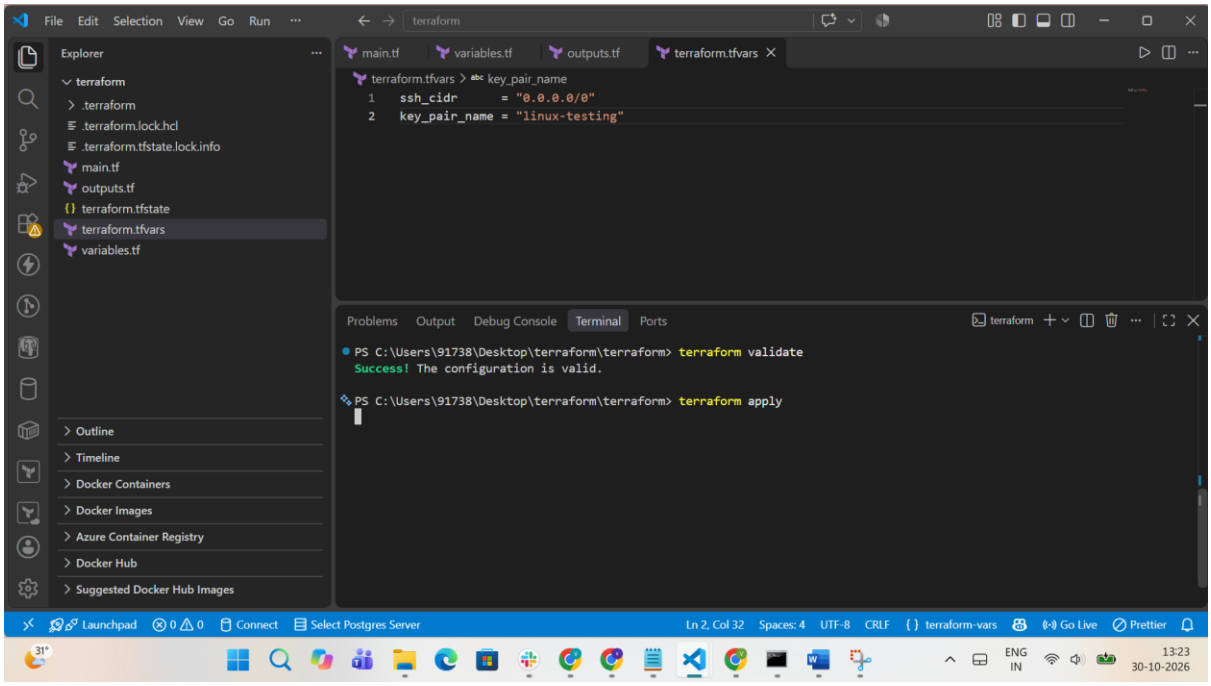
terraform apply

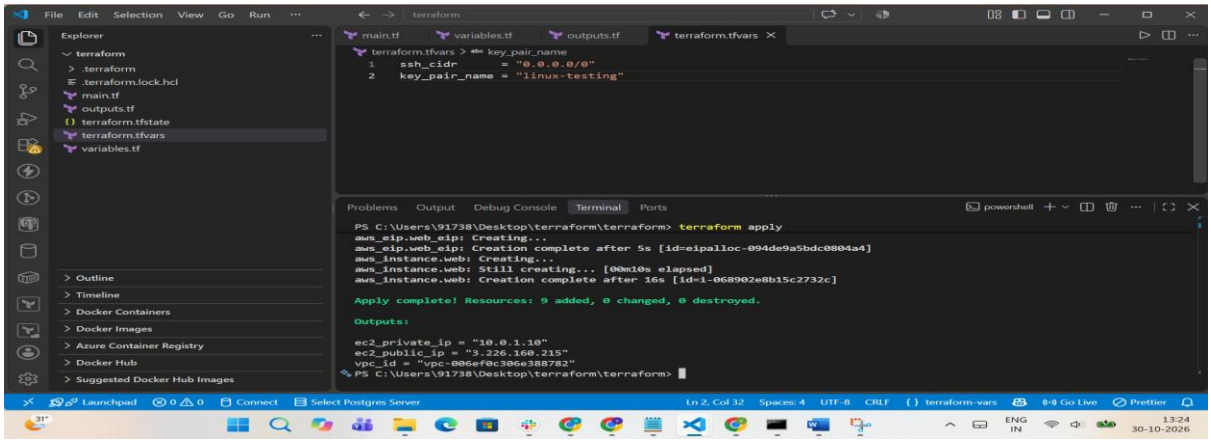
Enter yes when prompted.

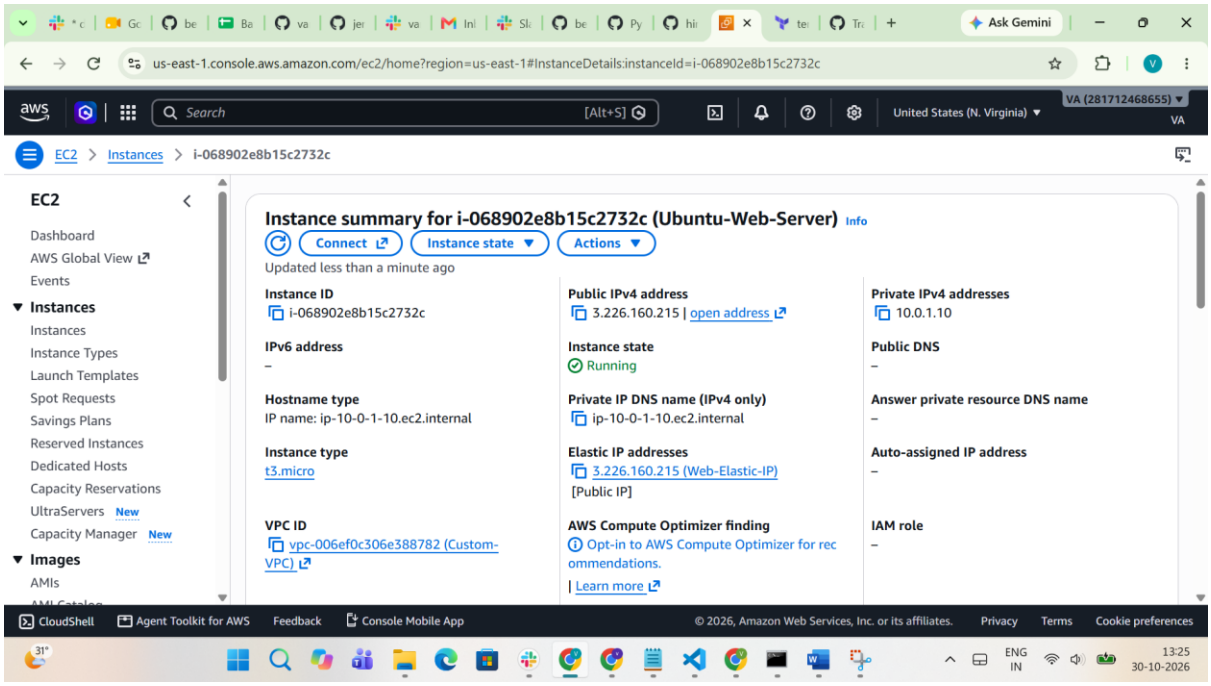
Verify Apache2 through a web browser:

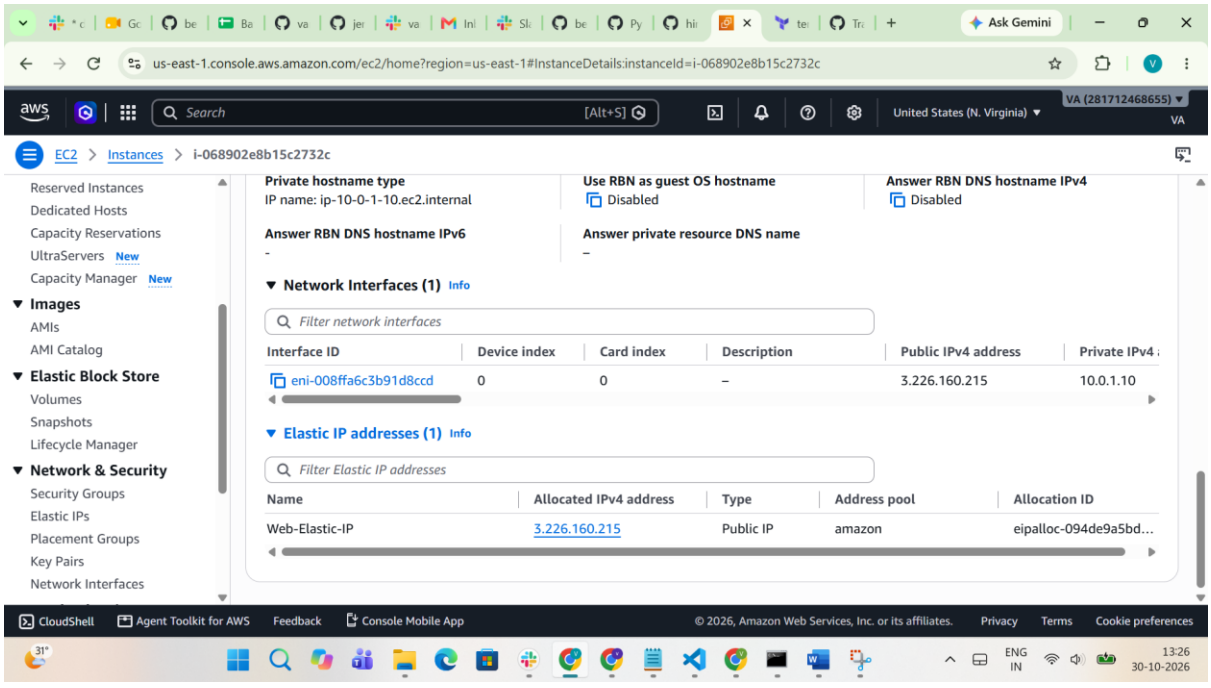
1. Copy the Elastic IP address associated with the EC2 instance.
2. Open a web browser.
3. Enter the following URL in the address bar:

   http://<Elastic-Public-IP>
4. Press Enter.
5. Verify that the Apache2 Ubuntu Default Page is displayed.
6. This confirms that Apache2 is successfully installed, running, and accessible over HTTP.











Welcome to Apache Server - Terraform

